

Technical University of Munich
School of Computation, Information and Technology

Embedded Systems, Cyber-Physical Systems and Robotics
Final Project Report

Text-to-Braille Translation Device

A Wearable Assistive Device for Real-Time Tactile Reading

Team Members:

Ansar Ahmad – 03798147
Luca Aquilani – 03802086
Arihan Gupta – 03803385
Alexandru Oprea – 03802652
Timofei Rusanov – 03820591
Richard Simorangkir – 03803656
Ebrahim Udaipurwala – 03802609
Marcel Zuhmann – 03804826

Course:

Embedded Systems, Cyber-Physical Systems and Robotics

Submission Date:

09.09.2026

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Related Work	2
1.3	Project Objectives	3
1.4	Project Scope and Limitations	3
2	Background	4
2.1	Braille	4
2.2	Optical Character Recognition (OCR)	5
2.3	Image Preprocessing	6
2.4	Solenoids	6
3	System Design and Architecture	6
3.1	Design Philosophy	6
3.2	Hardware Components	7
3.3	Circuitry	7
3.4	Mechanical Enclosure	9
3.4.1	Tactile Actuation	9
3.4.2	Scanning Subsystem	10
4	Implementation	11
4.1	Pipeline Architecture	12
4.2	OCR Backend Abstraction	12
4.3	Braille Translation and Tactile Output	13
5	Testing and Findings	14
5.1	Test Methodology	15
5.2	Development-Time Testing Tools	15
5.3	Results	15
5.4	Interpretation	16
6	Evaluation and Discussion	16
6.1	System Strengths and Performance	16
6.2	Limitations and Improvements	17
7	Conclusion and Future Work	18

Abstract

This report presents a hand-mounted device that converts printed text into real-time tactile Braille output for visually impaired users. Built around a Raspberry Pi 4, the system captures text with a camera module, recognizes it through a local or cloud-based OCR pipeline, translates it into Braille cells, and renders the output through six micro-solenoid actuators positioned against the user’s fingertip. Initial testing confirms a functional end-to-end sense-compute-actuate pipeline, achieving up to 99.4% OCR accuracy and an average scan-to-output latency of 1.7 seconds, while highlighting solenoid overheating and target user validation as the primary remaining challenges.

1 Introduction

1.1 Motivation

Printed text, like menus, handouts and books, is largely inaccessible to visually impaired individuals without external assistance. The most common existing workaround is text-to-speech scanning, either via dedicated hardware or smartphone applications that read text aloud. However, audio output may be undesirable in certain settings, and listening provides an inherently different experience from reading. Braille lets visually impaired users perceive spelling, punctuation, capitalization, formatting, and spacing directly.

While Braille displays provide such a reading experience, they cannot be used for printed text that is not available in the required digital format. Furthermore, Braille education faces a series of longstanding structural problems, resulting in a sharp decline in Braille literacy [8]. We aim to create a prototype device which allows visually impaired users to read any text and can be used by Braille readers and learners of all proficiency levels, making printed content more accessible and serving as a potential tool in Braille education.

1.2 Related Work

Several independent projects and educational resources directly informed the direction of this project. Brett Perl proposed a similar hand-worn text-to-Braille device using a smartphone’s camera for OCR, rendering output via actuated pins [5]. A separate concept, presented in a YouTube demonstration, explores a comparable finger-mounted Braille actuation mechanism [1]. Brizio demonstrated a simpler ASCII-to-Braille translator using an Arduino and six solenoids, without an OCR or camera-based input stage [2]. Similarly, Finio’s Science Buddies project guide presents an educational, single-cell refreshable Braille display built with Arduino, solenoids, and MOSFET drivers, accompanied by a demonstration video [4, 7]. These projects establish solenoid-based Braille cell actuation as a viable, low-cost approach, but remain proof-of-concept prototypes rather than commercially viable products. The relatively limited number and scope of existing solutions for this problem motivated us to make our own contribution and allowed us to create a novel solution which has never been built before.

1.3 Project Objectives

The core idea of this project is to deliver a device that translates printed text into physical Braille output, rendered onto the user's fingertip in near real-time. A handheld camera module allows the user to capture text as a single still image of a page or section. The user's other hand rests on the Braille actuation module, with the index finger positioned over the six pins that form a single Braille cell. Once the captured image is processed, the corresponding pins are actuated via a wired connection to the actuation module.

The project objectives are grouped into three categories:

- **Design and portability objectives:** Separate the bulkier working components including compute and imaging electronics from the user-interactive finger module, keeping the actuation module's component count and physical footprint small, so that the device can be worn and carried without external tethering while only the actuation surface makes contact with the fingertip.
- **Processing objectives:** Achieve a close to real-time OCR pipeline, minimizing the delay between triggering a scan and receiving tactile output.
- **Social objectives:** Provide a discreet, affordable tool that gives visually impaired users independent access to printed text in everyday public and private settings, and serve as an educational aid offering different difficulty levels for Braille learners.

1.4 Project Scope and Limitations

Scope This project is developed as a functional proof-of-concept rather than a certified, production-ready product. The OCR pipeline is configured for English-language recognition of printed Latin-script text on still images. A continuous OCR pipeline working directly on the video stream was planned, with the intention of allowing the user to move the device over text by hand and receiving instant feedback rather than taking an image. However, this method turned out to be infeasible within the scope of our project, given hardware constraints and the difficulty of moving the device with enough precision to capture machine-readable input.

Limitations The following limitations are consequences of design choices made within the project's timeframe and budget, and are examined quantitatively in Section 6:

- Local on-device OCR is limited by the Raspberry Pi 4's compute; a cloud-based OCR API is offered as a faster alternative, and button-triggered single-capture scanning is preferred over continuous scanning for reliability.
- The mini solenoids overheat under prolonged continuous use, limiting how long the device can be operated continuously.

2 Background

This section introduces the concepts directly relevant to the system’s design: the structure of Braille as a tactile encoding scheme, the terminology used in optical character recognition throughout this report, and the operating principle of the solenoid actuators used for tactile rendering.

2.1 Braille

Braille is a tactile writing system in which each character is represented by a cell of up to six raised dots arranged in a 2×3 grid, as illustrated in Figure 1, giving $2^6 = 64$ possible dot combinations. Two main forms of Braille exist.

Uncontracted Braille (Grade 1) maps each cell directly to a single letter, numeral, or punctuation symbol, as illustrated by the direct mapping seen in Figure 2.

Contracted Braille (Grade 2) additionally uses reserved cell combinations to represent common letter groups or whole words, reducing the physical space and reading time required for a given text, at the cost of greater encoding complexity. Some abbreviations used in contracted Braille are illustrated in Figure 3.

Braille cell dimensions are additionally standardized to remain consistent with typical fingertip tactile sensitivity; the actuator layout in this project follows a dot spacing of 2.5 mm and a dot tip diameter of 1.6–1.7 mm, consistent with common tactile Braille dimensions, used as the reference for the pin housing design described in Section 3.4.1.

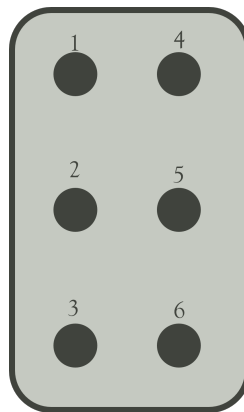


Figure 1: Standard dot numbering convention for a Braille cell, used throughout this report (e.g., the GPIO pin mapping in Table 2).

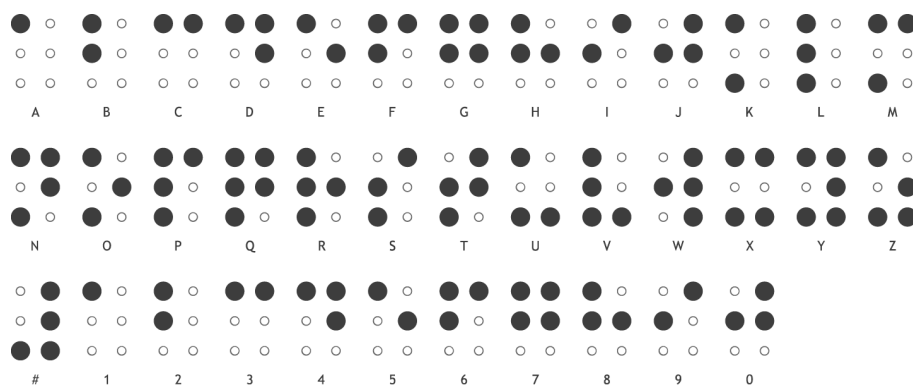


Figure 2: Uncontracted Braille Chart. Source [6]

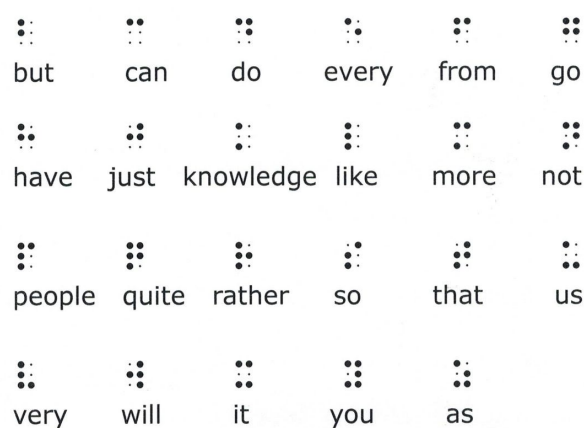


Figure 3: Common Braille contractions

2.2 Optical Character Recognition (OCR)

Optical character recognition (OCR) is the process of converting an image containing text into machine-encoded characters. This report uses the following terminology: **preprocessing** refers to image operations applied before recognition to improve reliability (detailed in Section 2.3); **recognition** refers to the classification of detected image regions into specific characters; and **recognition accuracy** refers to the proportion of correctly identified characters relative to the ground truth text. Each detected character or word is additionally associated with a bounding box, the rectangular image region it occupies; because OCR engines typically return results in detection order rather than left-to-right reading order, bounding-box coordinates are needed to reconstruct the original text order. OCR can be performed either locally, on-device, trading accuracy or speed for independence from network connectivity, or via a cloud API which typically provides higher accuracy, at the cost of network latency and a connectivity requirement. This project evaluates both approaches, described in Section 4.2. These terms are used throughout Sections 4 and 6 when describing the OCR pipeline and its evaluation.

2.3 Image Preprocessing

Preprocessing transforms a raw camera image into a form better suited for character recognition, and typically includes operations such as grayscale conversion, contrast enhancement, and binarization. These operations are particularly relevant to this project because the hand-mounted camera cannot guarantee consistent lighting or a fixed distance from the text, unlike a flatbed scanner. The specific preprocessing steps applied in this implementation are detailed in Section 4.

2.4 Solenoids

A solenoid is an electromechanical actuator consisting of a coil of wire wound around a movable ferromagnetic core. Passing current through the coil generates a magnetic field that displaces the plunger, producing linear motion; removing the current allows the plunger to return to its resting position, typically assisted by a spring (Figure 4). This binary extend/retract behavior maps naturally onto Braille dot rendering, since each of the six dot positions in a Braille cell is itself a binary state (raised or not raised), making solenoids well suited to this project’s actuation requirements. Because solenoids draw substantially more current than a microcontroller’s digital output pins can supply directly, dedicated driver circuitry is required to switch this current under low-power digital control, as described in Section 3.2.

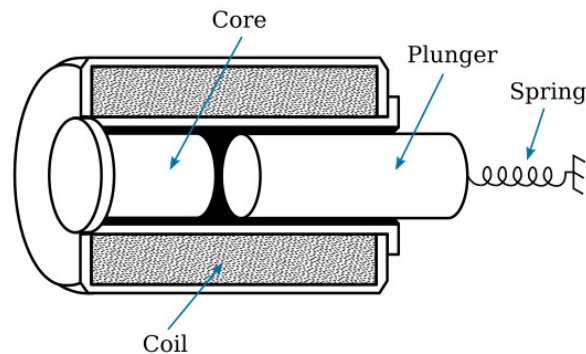


Figure 4: Basic operating principle of a solenoid actuator. Source: [3]

3 System Design and Architecture

This section details the physical and electronic architecture of the prototype. It outlines the core hardware components, electrical circuitry, and custom mechanical enclosures designed to reliably translate visual text into tactile Braille output while maintaining ergonomic usability.

3.1 Design Philosophy

The architecture was driven by a small set of practical priorities: minimizing the number and size of components to maximize portability, and keeping the compute and control logic centralized around a single, well-supported platform to simplify integration within the project timeframe.

Consistent with the original design objective (Section 1), the bulkier working components, i.e. the compute, driver boards, and power electronics, are physically separated from the user-interactive finger module: they are housed in a hand-mounted unit, connected to the finger actuation surface via a set of extended wires, so that only the six solenoid pins make direct contact with the fingertip, as illustrated in Figure 6a. This wiring introduces a small amount of added bulk and reduces the flexibility of finger movement relative to a fully integrated finger module, a trade-off revisited in Section 6.

3.2 Hardware Components

Table 1 summarizes the core hardware components used in the system.

Table 1: Core hardware components

Component	Role
Raspberry Pi 4 Model B	Main compute unit: runs OCR, translation, and actuation control
Raspberry Pi Camera Module (2028×1520)	Image acquisition for OCR input
Pololu DRV8835 (×3)	Dual-channel motor drivers, controlling the 6 solenoids (2 per board)
Breadboard (30×10 main grid, 2×25×2 power rails)	Interconnects the driver boards, solenoids, and power distribution, minimizing the additional point-to-point wiring
5V mini push-pull solenoids (×6)	Braille dot actuation, one per dot position
USB-C Breakout Board	Power supply for compute and actuation subsystems

The Raspberry Pi 4 was chosen as the compute platform for its sufficient processing power to run OCR at acceptable latency, native camera module support, and general-purpose GPIO for interfacing with the driver boards, avoiding the need for a separate microcontroller. Three DRV8835 driver boards were used, each providing two independently controllable output channels, giving exactly six channels (one per solenoid) without requiring additional multiplexing logic. Each channel is driven directly by a GPIO signal from the Raspberry Pi, switching the higher current required by the solenoid coil without loading the GPIO pin directly.

3.3 Circuitry

Figure 5 shows the wiring between the Raspberry Pi GPIO header, the three DRV8835 driver boards, and the six solenoids, along with the shared power and ground distribution.

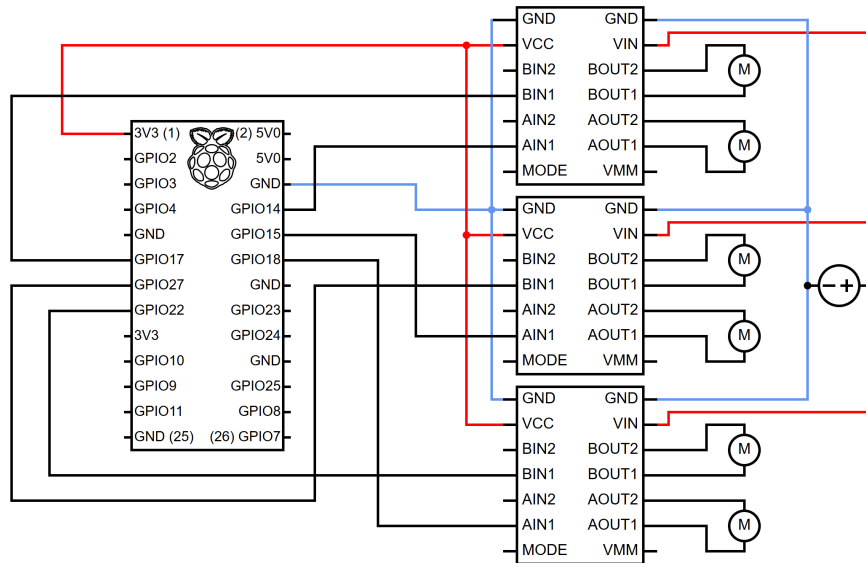
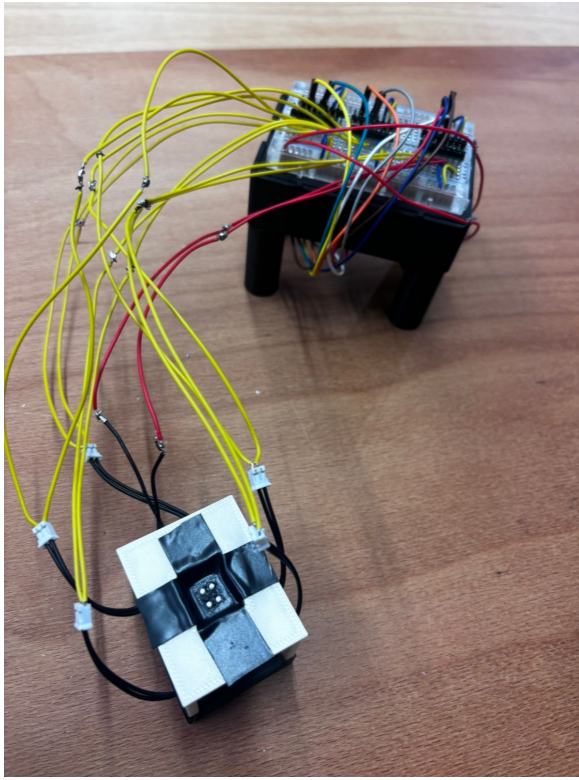


Figure 5: Circuit diagram of the actuation and power subsystem. Each DRV8835 board drives two solenoids; logic power (3V3, pin 1) and ground are shared across all three boards, while motor power (VIN) is supplied separately.

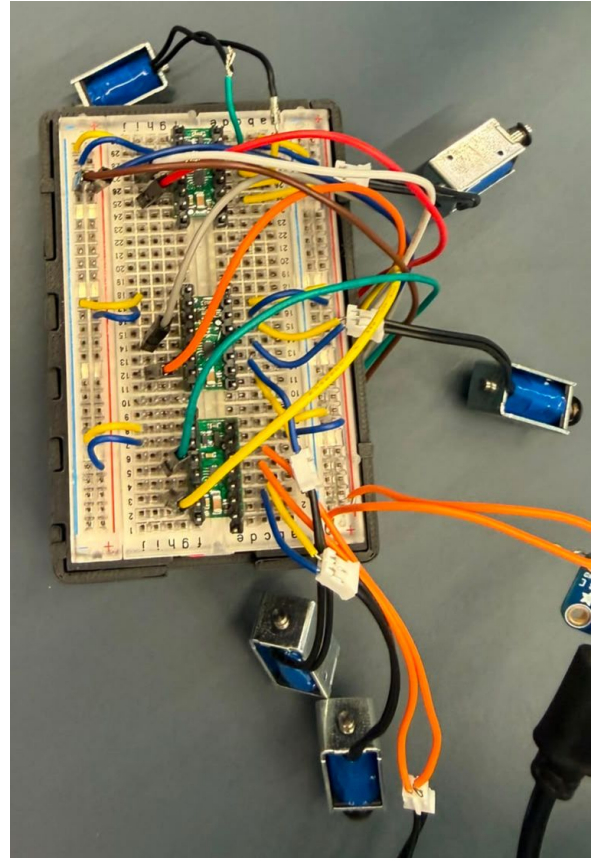
Table 2 lists the specific BCM GPIO pin driving each of the six Braille dot positions, corresponding to the GPIO connections shown in Figure 5.

Table 2: BCM GPIO pin mapping for each Braille dot position

Braille cell dot	BCM GPIO	Physical header pin
1	GPIO17	11
2	GPIO27	13
3	GPIO22	15
4	GPIO14	8
5	GPIO15	10
6	GPIO18	12



(a) Wrist/hand-mounted compute unit connected to the finger-worn actuation module via extended wiring.



(b) The six solenoids wired to the DRV8835 driver boards on the breadboard.

Figure 6: Physical realization of the hardware described in Table 1.

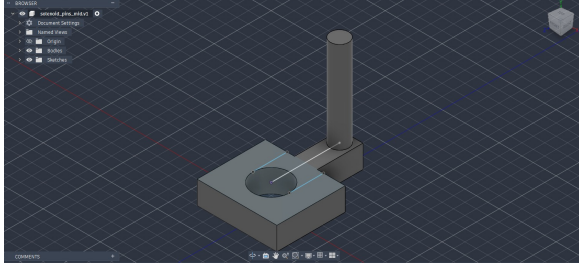
3.4 Mechanical Enclosure

To physically realize the decoupled architecture outlined in Section 3.2, a custom mechanical enclosure was designed to house the electrical components and facilitate safe user interaction. The design translates raw actuator movement into precise tactile feedback while completely isolating bare electronic circuitry from the user.

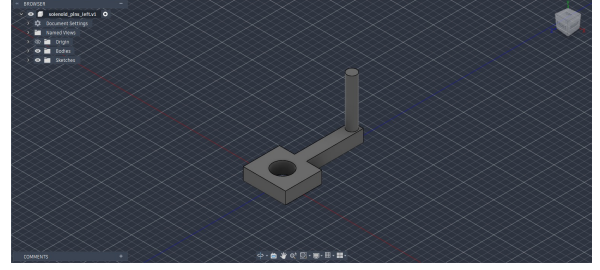
3.4.1 Tactile Actuation

The tactile Braille module integrates six discrete solenoids with a custom linkage and pin system (Figure 7). Custom mechanical linkages, designed in middle (Figure 7a) and side (Figure 7b) variants, transfer solenoid actuation into vertical tactile pin displacement.

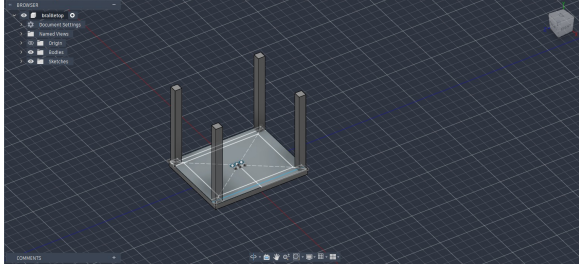
The upper interface plate features a standardized six-hole Braille cell arrangement (Figure 7c) to guide the vertical travel of each tactile pin (Figure 7d) and establish a stable surface for the user's finger. The mechanism is mounted within a recessed structural base (Figure 7e) that secures the solenoids and maintains mechanical alignment between the actuators and output pins in the complete assembly (Figure 7f).



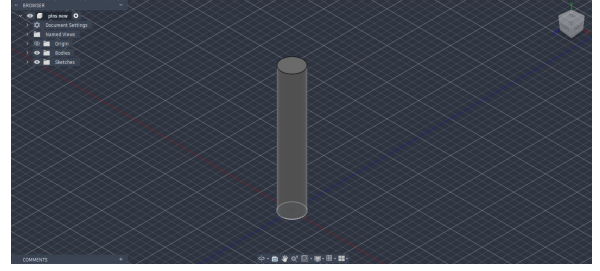
(a) Middle linkage variant.



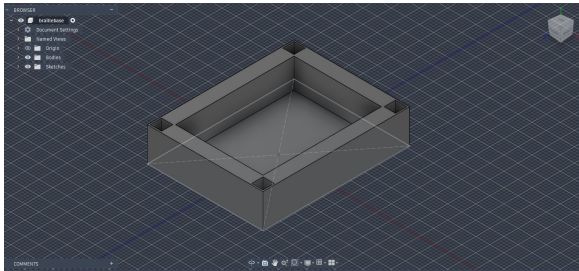
(b) Side linkage variant.



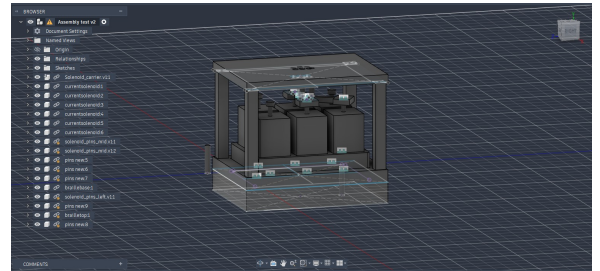
(c) Top plate establishing the structural alignment.



(d) Individual tactile pin geometry.



(e) Recessed base providing the structural foundation.



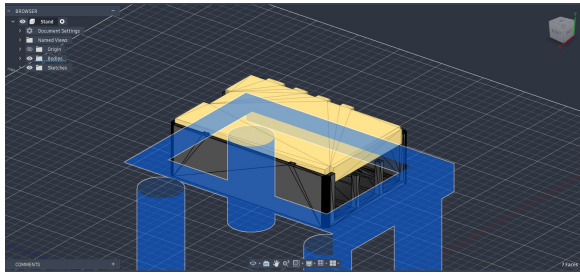
(f) Full CAD assembly concept with packaged solenoids.

Figure 7: Fusion 360 CAD models illustrating the kinematic translation from solenoid actuation to tactile Braille output.

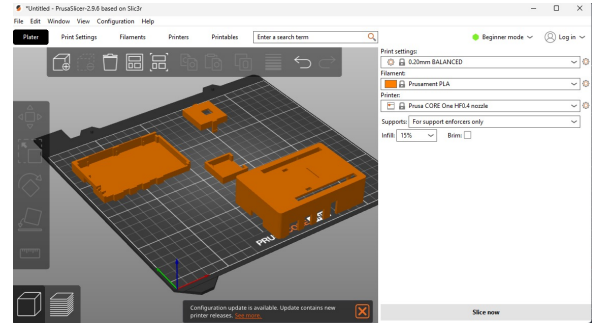
3.4.2 Scanning Subsystem

A dedicated enclosure houses the Raspberry Pi and positions the camera at a fixed angle for consistent text capture (Figure 8a), further ensuring actuator vibration does not degrade optical performance.

All enclosure components were manufactured via Fused Deposition Modelling (FDM) 3D printing and prepared using PrusaSlicer (Figure 8b). The realized physical prototype validates camera positioning inside its printed mount (Figure 9a) and integrates the solenoids, linkages, and top plate into a fully enclosed actuator unit (Figure 9b).



(a) Adapted Pi enclosure with an integrated camera mount.

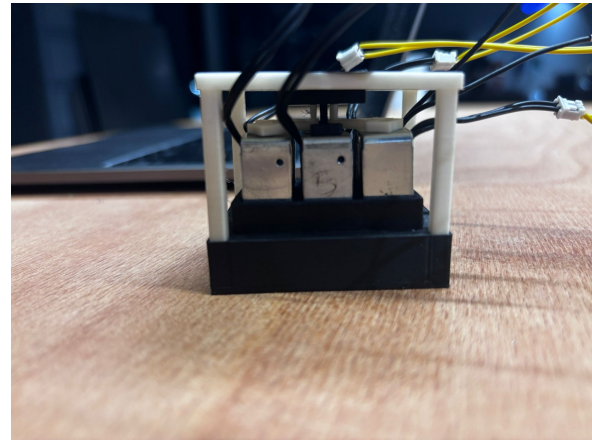


(b) Housing components arranged on the PrusaSlicer build plate.

Figure 8: Design and slicing preparation of the decoupled optical scanning module.



(a) Physical camera module fit validation within the printed housing.



(b) Fully wired actuator module with solenoids and linkages installed.

Figure 9: Realized physical prototypes demonstrating successful system integration.

4 Implementation

The reader is implemented entirely in Python and is organized as four loosely-coupled modules: capture and control (`reader_main.py`), optical character recognition (`ocr_backend.py`), Braille translation and actuation (`braille_controller.py`), and a diagnostic web visualizer (`solenoid_visualizer.py`). Each stage communicates with its neighbors through a narrow, well-defined interface, so that any stage can be swapped or reimplemented (e.g., changing OCR

engines, or moving from solenoids to a refreshable Braille cell) without touching the rest of the pipeline. The GPIO outputs described in this section drive the three DRV8835 boards and six solenoids introduced in Section 3.2.

4.1 Pipeline Architecture

At runtime the system executes a five-stage pipeline, shown in Figure 10: image capture, preprocessing, OCR, Braille translation, and tactile output. These stages map onto the classic cyber-physical sense-control-actuate loop: capture and preprocessing form the *sense* stage, OCR and Braille translation form the *control* stage, and tactile output forms the *actuate* stage. The capture loop itself is non-blocking: OCR and Braille playback are each dispatched on separate threads so that a slow recognition call or a multi-second tactile readout never stalls the camera or the keyboard trigger.

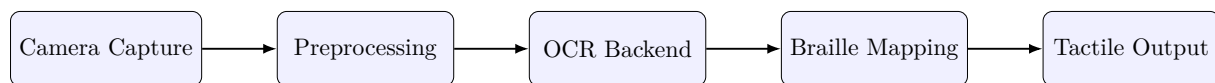


Figure 10: Software pipeline stages, from camera frame to tactile output.

Capture and triggering. A background thread listens for a keypress, so capture is user-triggered rather than continuous. The camera itself is configured with two simultaneous streams: a full-resolution `main` stream (2028×1520) used only at the moment of capture for OCR, and a low-resolution `lores` stream (320×240) continuously converted to grayscale and pushed to the live-view HTTP server, keeping the interactive preview cheap while preserving OCR quality.

Preprocessing. Before a captured frame is sent to either OCR backend, a short preprocessing step corrects common problems with handheld photos. First, the image is straightened: the program estimates how much the text is tilted and rotates the frame to make it level. Next, the lighting is evened out using an adaptive contrast adjustment, which brightens dim areas and darkens overly bright ones without washing out the whole image the way a simple global brightness fix would. Finally, if the captured frame is too small, it is enlarged using smooth interpolation, since both OCR engines recognize text more reliably when characters are larger. The image is deliberately kept in color/grayscale rather than converted to pure black and white for the cloud-based backend, since the OCR.space API already performs its own thresholding internally and tends to perform worse when given an already-binarized image.

4.2 OCR Backend Abstraction

The two OCR engines are unified behind a single abstract interface, `OCRBackend`, selected at startup via a factory function (`build_backend`). Both concrete backends return the same triple, (`text`, `edge_touched`, `error`), which keeps the capture loop free of any backend-specific branching:

- **text** — recognized text collapsed to a single line;

- **edge_touched** — a boolean heuristic flag indicating a detected word’s bounding box lies within `EDGE_MARGIN_PX` (20 px) of the frame border, used to warn the user that text may have been cropped and that they should reframe the shot;
- **error** — a non-empty diagnostic string on failure, so the caller is informed that `text` is inaccurate.

Local backend (RapidOCR). `LocalRapidOCRBackend` runs detection and recognition models on-device using mobile-sized model variants (`ModelType.MOBILE`) and an English-only recognition language. Because OCR engines return bounding boxes in detection order rather than reading order (Section 2), the recognized text fragments are explicitly re-sorted by the minimum x -coordinate of their bounding box before being joined:

Listing 1: Reconstructing left-to-right reading order from unordered OCR detections.

```

1 items = []
2 for i in range(len(res.bboxes)):
3     t = (res.txts[i] or "").strip()
4     if not t:
5         continue
6     xs = np.array(res.bboxes[i][:, 0])
7     x_min, x_max = float(xs.min()), float(xs.max())
8     if x_min <= EDGE_MARGIN_PX or x_max >= frame_w - EDGE_MARGIN_PX:
9         edge_touched = True
10    items.append((x_min, t))
11 items.sort(key=lambda z: z[0])
12 return " ".join(t for _, t in items), edge_touched, ""

```

Cloud backend (OCR.space). `OCRSpaceBackend` encodes the captured frame as JPEG and submits it as a multipart form upload to the OCR.space REST endpoint, requesting the overlay-coordinates option so that per-word bounding boxes are returned alongside the parsed text. The same edge-margin heuristic is then applied to the overlay’s word coordinates to produce an `edge_touched` flag consistent with the local backend. Network and JSON decoding failures are caught explicitly and surfaced through the **error** field rather than raising, so a dropped connection degrades to a warning rather than crashing the capture loop.

4.3 Braille Translation and Tactile Output

`BrailleController` supports three translation modes, selectable at construction time or at runtime:

For the liblouis-backed modes, translation is run once over the complete recognized line rather than character-by-character: a contraction such as “the” or the `-ing` ending only exists as a single Braille cell in Grade 2 output once liblouis has visibility into the surrounding letters, so per-character translation (as used by the `naive` mode) is architecturally incompatible with contracted Braille and is reserved for the dependency-free fallback path.

Table 3: Braille translation modes

Mode	Description
naive	Direct, built-in one-to-one character-to-cell mapping (uncontracted, Grade 0); no external dependency.
grade1	liblouis Grade 1 Unified English Braille — every letter spelled out, but with correct punctuation/number handling via the <code>en-ueb-g1.ctb</code> table.
grade2	liblouis Grade 2 Unified English Braille — contracted output (e.g. whole-word and common-sequence contractions such as “the” and “ing”), matching what most modern embossers and refreshable displays produce, via <code>en-ueb-g2.ctb</code> .

Dot decoding algorithm. liblouis returns translated text as a string of Unicode Braille Pattern characters (block U+2800–U+28FF), where each of the low 6 bits of `codepoint - 0x2800` corresponds to one of the six dots in a cell. The controller decodes each cell with a constant-time bitmask test:

Listing 2: Decoding a liblouis Unicode Braille cell into the set of physically active dots.

```

1 _DOT_BITS = {1: 0x01, 2: 0x02, 3: 0x04, 4: 0x08, 5: 0x10, 6: 0x20}
2
3 def _unicode_cell_to_dots(self, cell_char):
4     bitmask = ord(cell_char) - 0x2800
5     return tuple(dot for dot, bit in _DOT_BITS.items() if bitmask & bit)

```

Codepoints that also set the dot-7/dot-8 bits (8-dot computer Braille or certain math tables) are detected but silently truncated to 6 dots, since the physical cell only actuates six solenoids.

Actuation and hardware abstraction. Each physical dot is mapped to a fixed BCM GPIO pin, listed in Table 2, and a single low-level routine, `_actuate_dots`, is shared by both the naive and liblouis playback paths, ensuring the two modes cannot diverge in how they drive hardware. When `RPi.GPIO` is unavailable (e.g., running on a development machine rather than the Pi), a drop-in `MockGPIO` class transparently logs pin state changes to the console instead of raising an import error, and every state change is mirrored into a shared `viz_state` dictionary consumed by the web-based solenoid visualizer — allowing the full software pipeline to be developed and demonstrated without the physical Braille cell attached.

5 Testing and Findings

This section evaluates the prototype’s core optical capture and software processing pipelines. Following the project’s testing protocol, the objective of the OCR accuracy test was to measure character recognition accuracy across realistic lighting conditions. Concurrently, the latency test was designed to measure end-to-end delay from trigger to first tactile output. These metrics are critical for validating the system’s real-world viability as a continuous reading aid.

5.1 Test Methodology

Testing was performed on printed paragraphs of text, rather than isolated words or single sentences, to reflect the intended real-world use case of reading continuous printed text. Five distinct paragraph samples (labeled C1–C5) were used, each printed on a separate card. Accuracy was evaluated under two lighting conditions: normal light and dim light. Each of the five cards was scanned once per lighting condition, with five trials per condition, yielding ten trials in total. For each trial, the OCR output text was compared character-by-character against the known ground-truth text of the card, and accuracy was computed as the proportion of correctly recognized characters out of the total character count. Reported accuracy per condition is the mean across its five trials.

Scan-to-output latency was measured as the sum of two stages: camera capture time and combined OCR plus network processing time. Five latency trials were logged programmatically. The results below reflect only the cloud backend using the OCR.space API.

5.2 Development-Time Testing Tools

Prior to physical hardware integration, two live HTTP streams introduced in Section 4, namely the camera live-view stream and the simulated Braille stream, were used to validate capture, OCR, and translation behavior independently of the tactile output stage. Because both streams are driven by the same shared state used by the physical GPIO actuation path, behavior verified through them required no separate re-verification once moved to the physical device.

5.3 Results

Accuracy Table 4 summarizes mean OCR recognition accuracy (cloud backend) by lighting condition, averaged over the five paragraph samples tested in each condition.

Table 4: Mean OCR recognition accuracy (%) by lighting condition (cloud backend)

Condition	Samples	Average accuracy (%)
Normal light	5	99.4
Dim light	5	96.4

Latency Table 5 summarizes scan-to-output latency (cloud backend) across five trials, broken down into camera capture time and combined OCR/network processing time.

Table 5: Scan-to-output latency by stage (cloud backend, ms)

Metric	Camera capture	OCR + network	Total latency
Average	182	1,515	1,697
Minimum	169	1,370	1,551
Maximum	201	1,636	1,810

5.4 Interpretation

With the cloud backend, recognition accuracy remained high in both tested conditions, dropping only modestly from 99.4% under normal lighting to 96.4% under dim lighting. This suggests the cloud OCR engine is reasonably robust to reduced lighting for printed text at this scale. The observed degradation manifested purely as isolated character-level confusions (such as interpreting 1 as I or o as 0 in dim light, and B as P in normal light) rather than the failed recognition of entire words or lines.

OCR and network processing account for the large majority of end-to-end latency (1,515 ms of 1,697 ms on average, roughly 89%), with camera capture contributing a comparatively small and consistent 182 ms. Given that this figure reflects the cloud backend specifically, part of this latency is attributable to the network round-trip rather than recognition compute alone.

6 Evaluation and Discussion

This section evaluates the performance of the integrated text-to-Braille translation device against its architectural and functional goals. The prototype’s performance is analyzed through its core operational achievements alongside its computational, thermal, and user-validation constraints.

6.1 System Strengths and Performance

The prototype demonstrates several robust architectural advantages, combining high-accuracy optical recognition with a versatile translation pipeline and an ergonomically decoupled hardware layout. Table 6 details these primary strengths alongside empirical evidence gathered during physical testing.

Table 6: Core system strengths, functional benefits, and empirical validation.

Feature and Functional Benefit	Empirical Evidence
High Accuracy & Predictable Latency: Delivers highly reliable text recognition across varying ambient conditions with consistent processing speed.	Achieved 99.4% character accuracy under normal light and 96.4% in dim light, with an average end-to-end latency of 1,697 ms.
Multi-Grade Braille Literacy Support: Accommodates users with varying degrees of Braille literacy, from beginner learners to proficient readers.	Embedded translation software dynamically processes and outputs Grade 0 (raw characters), Grade 1, and Grade 2 (contracted) Braille.
Decoupled Physical Architecture: Prevents actuator vibration from degrading camera focus while minimizing physical payload on the user's reading finger.	Compute unit, driver boards, and power routing are isolated in a hand-mounted enclosure, leaving only the 6-pin actuator module on the finger.

6.2 Limitations and Improvements

Despite achieving its core functionality, physical and computational testing revealed key constraints in thermal dissipation, processing throughput, and user evaluation. Table 7 outlines these limitations and outlines concrete architectural improvements for future iterations.

Table 7: Key observed limitations and proposed architectural improvements.

Observed Limitation	Proposed Improvement
Thermal Saturation Under Extended Use: Continuous coil holding current in the 5V solenoids and sustained CPU load on the Raspberry Pi 4 generate substantial heat during prolonged reading sessions.	Implement Pulse Width Modulation (PWM) on the DRV8835 driver channels to scale down current after initial pin elevation to decrease solenoid overheating, and add active heat-sinking/fan cooling to the Pi housing.
Compute Constraints for Local OCR & Continuous Scanning: The Raspberry Pi 4 lacks the processing throughput needed to run local engine inference (e.g., Tesseract) or real-time continuous video stream scanning without severe latency spikes.	Upgrade the central compute platform to a higher-performance edge accelerator (e.g., Raspberry Pi 5 or Nvidia Jetson Nano) capable of handling local parallelized OCR pipelines.
Lack of Target User Validation: The device was evaluated strictly on technical parameters (character accuracy, stage latencies, mechanical pin movement) without formal testing by visually impaired or blind Braille readers.	Conduct qualitative user-experience studies with target end-users to evaluate real-world tactile clarity, dot height legibility, ergonomic comfort, and reading speed.

7 Conclusion and Future Work

This project successfully demonstrated the technical viability of a modular text-to-Braille translation device, effectively bridging the gap between digital text recognition and physical tactile output. By implementing a decoupled hardware architecture and a multithreaded software pipeline, the prototype achieved high character recognition accuracy and predictable latency while seamlessly translating captured text into multi-grade Braille. Furthermore, the custom FDM-printed enclosures and linkage mechanisms validated the feasibility of safely isolating high-current electronic components from the user while delivering precise mechanical actuation directly to the fingertip.

While the current architecture satisfies its foundational proof-of-concept objectives, expanding its practical utility highlights three core avenues for future development. First, enabling natural, real-time left-to-right scanning requires transitioning from cloud-based OCR—which introduces a ~1.6 s latency bottleneck—to a highly optimized local OCR engine running on a more capable compute unit. Second, integrating finger-guidance feedback through sensor-assisted enclosure modifications will allow users to align the camera with text lines independently. Finally, the software pipeline can be generalized to support handwritten text, diverse typographic fonts, and

non-Latin scripts by integrating broader OCR models and expanded Braille translation tables.

Ultimately, technical benchmarking serves only as a precursor to human-centered validation. The next phase of development must shift from laboratory testing toward qualitative trials with blind and visually impaired readers to evaluate real-world tactile clarity, dynamic reading speeds, and long-term ergonomic comfort, ensuring the system evolves from an engineering prototype into a practical assistive tool.

References

- [1] Finger-worn braille device concept, 2023. <https://youtu.be/v1g8lJbK9u4>.
- [2] Cesare Brizio. ASCII-Braille real-time translation via Arduino. Arduino Project Hub, 2019. <https://projecthub.arduino.cc/CesareBrizio/ascii-braille-real-time-translation-via-arduino-3c90dd>.
- [3] EC Studio Systems. Solenoid actuators – basic electronics. Electric Circuit Studio, 2021. <https://ecstudiosystems.com/discover/textbooks/basic-electronics/sensors-and-actuators/solenoids/>.
- [4] Ben Finio. Make an automatic Braille display to help blind people. Science Buddies, 2025. https://www.sciencebuddies.org/science-fair-projects/project-ideas/Elec_p109/electricity-electronics/refreshable-braille-display.
- [5] Brett Perl. Braille, 2024. <https://www.brettperl.com/braille>.
- [6] PharmaBraille. The Braille alphabet. PharmaBraille, 2020. <https://www.pharmabraille.com/pharmaceutical-braille/the-braille-alphabet/>.
- [7] Science Buddies. Refreshable Braille display with Arduino. YouTube, 2024. <https://www.youtube.com/watch?v=D0-yi5dm2qw>.
- [8] Arielle Michal Silverman and Edward C. Bell. The association between Braille reading history and well-being for blind adults, 2018. <https://nfb.org/images/nfb/publications/jbir/jbir18/jbir080103.html>.